Urban Sound Event Detection on RK3588 Using Pretrained YAMNet: Taxonomy Alignment, Spectral Masking, and Temporal Postprocessing

ELEC5305 Acoustic and Speech Processing, School of Electrical and Computer Engineering, The University of Sydney

Author: Mingze Li (Student ID: 550374130) | **Supervisor:** Prof. Craig Jin | **Date:** November 2026
Project Website: https://kolentossa.github.io/elec5305-project-550374130/ | **GitHub Repository:** https://github.com/kolentossa/elec5305-project-550374130

Abstract

Running sound event detection (SED) on an embedded board requires converting short-time frame or patch scores from a pretrained audio model into stable event start and end times. In this project, I test urban sound event detection on the **Rockchip RK3588 NPU (Embedfire LubanCat-5 V2)** using Google's pretrained **YAMNet** classifier (0.96 s patch window, 0.48 s hop, 64-band log-Mel spectrogram frontend). Instead of retraining the CNN weights, this work focuses on one research question: *how should patch-level class scores from pretrained YAMNet be mapped across event taxonomies and temporally integrated to balance false alarms, event fragmentation, and onset/offset detection delay on an embedded RK3588 NPU platform?* I first checked numerical consistency between Windows ONNX CPU, RK3588 ARM64 CPU, and FP16 RKNN NPU execution on 3.0 s audio windows (MAE = 1.98e-3, 15.3 ms per window on NPU Core 0), and wrote a 2.4 s sliding-window script that drops the 6th zero-padded patch on each window. In my initial 6-scene pilot (56 s, 119 patches), both CPU and NPU gave 100.0% micro precision and 85.7% micro recall, but showed three main issues: (1) the *car_horn* score dropped from 0.8765 to 0.0777 when mixed with a *siren*, (2) causal exponential moving average (EMA) smoothing extended interval offsets by +0.48 s, and (3) non-stationary classes suffered from mid-event score dips. In the second iteration (17 scenes, 178 s across separate ESC-50 Fold-3 validation and Fold-4 test sets), I ran controlled tests on both Windows CPU and RK3588 NPU: (1) an isolated -6 dB half-gain control (*horn_half*) reached a peak NPU score of 0.8296, confirming that the 0.0777 drop in *overlap* is caused by **spectral masking in the 64-band log-Mel spectrogram** rather than amplitude scaling; (2) grouping parent and child AudioSet engine classes raised *engine_idling* segment recall from 66.7% to 83.3% (Segment F1 from 0.800 to 0.909) and cut fragmentation in half (2.67 to 1.33); and (3) validation-tuned dual-threshold hysteresis (theta_on = 0.20, theta_off = 0.06) gave the highest test Segment F1 (0.864 on RK3588 NPU, 0.878 on CPU) and reduced fragmentation from 1.53 to 1.13 without adding the +0.47 s onset delay or +0.66 s offset tail seen under heavy causal EMA smoothing (alpha = 0.3). Across 87 three-second windows, RK3588 NPU Core 0 averaged 20.83 ms per 2.4 s step (RTF = 0.00868, about 115x faster than real time).

1. Introduction and Research Question

1.1 Problem Context and Edge Acoustic Monitoring

Detecting urban acoustic events such as emergency sirens, car horns, street machinery, or dog barks directly on edge hardware avoids streaming raw audio to a remote server and keeps response latency low. Pretrained audio classifiers such as Google's **YAMNet** (Hershey et al., 2017) provide a practical starting point for embedded deployment: trained on over two million 10-second clips from **AudioSet** (Gemmeke et al., 2017) using depthwise-separable MobileNetV1 convolutions (Howard et al., 2017), YAMNet maps 16 kHz mono audio into 521 audio event probabilities every 0.48 s with 3.75 million parameters.

However, using a clip-level audio tagger for continuous **sound event detection (SED)** introduces three signal-processing problems. First, acoustic energy inside a single event (such as an idling engine or repeated dog barking) fluctuates across consecutive 0.48 s hops, so a fixed threshold often turns on and off mid-event (**event fragmentation**). Second, urban scenes often contain overlapping sources: when a loud sweeping sound (such as a siren) overlaps in time and frequency with a harmonic sound (such as a car horn), logarithmic compression in the 64-band Mel frontend alters the relative feature pattern passed to the CNN. Third, running the model on the **Rockchip RK3588 NPU** in 16-bit floating-point (FP16) precision on fixed 3.0 s waveform windows requires handling boundary zero-padding and checking whether FP16 rounding changes threshold decisions.

1.2 Single Research Question

Following the course workflow (defining a concrete question, reproducing an existing open-source baseline, diagnosing its failure modes, and testing targeted modifications), my project addresses one specific research question:

> **Research Question: How should short-time patch-level class scores p_c(m) (0.96 s window, 0.48 s hop) from pretrained YAMNet be mapped across event taxonomies and temporally integrated to balance false alarms, event fragmentation, and onset/offset detection delay on an embedded RK3588 NPU platform?**

1.3 Project Stages

- **Stage 1 (Baseline Reproduction & Cross-Backend Check):** Convert Rockchip's exported *yamnet_3s.onnx* into an FP16 RKNN model (*yamnet_3s.rknn*), check numerical agreement across Windows x86_64 CPU, RK3588 ARM64 CPU, and RK3588 NPU Core 0, and write a 2.4 s sliding-window runner that drops the 6th zero-padded patch.
- **Stage 2 (Initial 6-Scene Pilot & Error Diagnosis):** Test six controlled urban scenes (56 s total) comparing fixed thresholding (theta = 0.20) with causal EMA smoothing (alpha = 0.5). Diagnose why *car_horn* dropped from 0.8765 to 0.0777 in the *overlap* scene and why EMA delayed interval endings by +0.48 s.
- **Stage 3 (Taxonomy Grouping, Masking Control & Hysteresis Study):** Map all 10 URBAN-SED classes, build separate Validation (Fold 3, 56 s) and Test (Fold 4, 122 s) splits with measured -35 dB acoustic onset/offset boundaries, run an isolated half-gain control (*horn_half*) to separate amplitude scaling from log-Mel spectral masking, test hierarchical engine class grouping, and compare seven temporal decision rules on Windows CPU and RK3588 NPU.

2. Literature Review and Theoretical Background

2.1 Time-Frequency Audio Representations and Log-Mel Masking

In YAMNet's frontend (Hershey et al., 2017), a 16 kHz mono waveform x[n] is split into 25 ms frames (N = 400 samples) with a 10 ms hop (H = 160 samples, 60% overlap), multiplied by a periodic Hann window w[n] = 0.5 - 0.5 cos(2*pi*n/N), and transformed via a 512-point discrete Fourier transform (DFT). The one-sided magnitude spectrum abs(X[t, k]) is mapped onto M = 64 triangular Mel filterbank bins spanning f_min = 125 Hz to f_max = 7500 Hz, where the Mel scale m(f) = 2595 log10(1 + f/700) reflects the approximate frequency resolution of human hearing. Each Mel band energy E_mel[t, b] is then compressed using a stabilised natural logarithm:

```
S_log[t, b] = log( E_mel[t, b] + epsilon ), where epsilon = 0.001
```

An important property of Eq. (1) is how it responds to **linear gain scaling** versus **two-source additive mixing**. If a single isolated source x_2[n] is scaled by a linear gain g_2 = 0.5 (-6.02 dB), its Mel energy scales by g_2^2 = 0.25, which (for active bands where E_2[t, b] >> epsilon) shifts S_log[t, b] by a constant offset log(0.25) while keeping the spectral shape across bands intact. In contrast, when two uncorrelated sources x_1[n] (such as a siren) and x_2[n] (such as a car horn) overlap in the same Mel band b, the combined log-Mel value is:

```
log( g_1^2 E_1[t,b] + g_2^2 E_2[t,b] + eps ) ~ log( g_1^2 E_1[t,b] + eps ) + log( 1 +
g_2^2 E_2[t,b] / (g_1^2 E_1[t,b] + eps) )
```

Whenever the stronger source satisfies g_1^2 E_1[t, b] >> g_2^2 E_2[t, b] in shared mid-frequency Mel channels, the second term becomes very small: the dominant source saturates those Mel bins and masks the harmonic contour of the weaker source. I test this directly in Section 4.3.

2.2 Pretrained Audio Tagging with MobileNetV1 (YAMNet) and Class Mapping

Every 48 STFT frames (0.48 s), YAMNet takes a 96 x 64 log-Mel patch (0.96 s of frame hops, covering 95 * 10 ms + 25 ms = 0.975 s of audio) and runs it through a 28-layer MobileNetV1 network (Howard et al., 2017; Hershey et al., 2017). Depthwise-separable convolutions factorise a 3 x 3 convolution into a 3 x 3 spatial depthwise filter followed by a 1 x 1 pointwise channel projection, producing a 1024-dimensional embedding vector e(m) after global average pooling. The output layer applies 521 independent sigmoids p_c(m) = 1 / (1 + exp(-z_c(m))). When adapting YAMNet to the 10-class **URBAN-SED** taxonomy (Salamon et al., 2014, 2017) using recordings from **ESC-50** (Piczak, 2015), mapping a class like *engine_idling* to a single narrow label (*Idling* [346]) misses score assigned to parent or sibling engine classes (*Engine* [337], *Medium engine* [342], *Heavy engine* [343]), which motivates grouping related AudioSet classes via maximum pooling.

2.3 Temporal Postprocessing and SED Evaluation Metrics

Turning patch scores p_c(m) into event intervals [t_start, t_end] requires temporal postprocessing (Cakir et al., 2017; Ebbers et al., 2022). Fixed thresholding has no filter delay but splits events whenever a single patch dips below threshold. Causal exponential moving average (EMA) smoothing applies a first-order low-pass filter p_bar_c(m) = alpha p_c(m) + (1 - alpha) p_bar_c(m-1), which reduces rapid score swings but adds filter delay tau_g = (1 - alpha)/alpha patches on both rising and falling edges. Dual-threshold **hysteresis** uses two thresholds (theta_on > theta_off): starting an event requires p_c(m) >= theta_on, while continuing an active event only requires p_c(m) >= theta_off. Following **Mesaros et al. (2016)**, I evaluate performance at both the **segment level** (patch-grid precision, recall, and F1) and the **event level** (event F1, fragmentation ratio, and onset/offset timing error).

2.4 Concepts Learned During This Project

While working on this project, I had to learn four practical concepts that were new to me:

1. **Boundary Zero-Padding in Fixed-Length NPU Models:** Understanding why a 3.0 s (48,000-sample) input to *yamnet_3s.onnx* outputs 6 patches where patch m = 5 (2.40 s to 3.375 s) includes 6,000 zero-padded samples, and why stepping the window by 2.4 s (keeping patches 0..4) avoids periodic boundary dips.
2. **Log-Mel Masking vs. Linear Gain Scaling:** Designing an isolated half-gain control (*horn_half*) to see why -6 dB amplitude scaling barely changes log-Mel scores (0.8765 -> 0.8296), whereas co-channel masking by a siren drops the score to 0.0777.
3. **Filter Delay in EMA vs. State Memory in Hysteresis:** Seeing why symmetric low-pass filtering trades fragmentation against onset/offset lag, whereas two-threshold hysteresis keeps onset timing unchanged.
4. **FP16 RKNN Model Conversion:** Using *rknn-toolkit2* to convert an ONNX model with fixed DFT convolution weights into an FP16 NPU binary on the RK3588 and checking how 1e-3 FP16 differences behave near thresholds.

3. Methodology and Experimental Design

3.1 Hardware Setup and Unpadded 2.4 s Sliding-Window Pipeline

All experiments were run on two platforms: (1) a **Windows 11 x86_64 PC** running *onnxruntime 1.24.4* (FP32), and (2) an **Embedfire LubanCat-5 V2 (Rockchip RK3588)** board (4x Cortex-A76 + 4x Cortex-A55, 6 TOPS NPU, Ubuntu 22.04 aarch64, RKNPU driver v0.9.8, *librknnrt.so* v2.3.2). Using *rknn-toolkit2 v2.3.2*, Rockchip's *yamnet_3s.onnx* model (which implements the 25 ms Hann-windowed STFT as fixed 1D/2D DFT convolution kernels) was converted into an FP16 NPU model (*yamnet_3s.rknn*) running on **NPU Core 0**.

For each 3.0 s window (48,000 samples at 16 kHz), the model outputs a (6, 521) score matrix where patch m in {0..5} covers [0.48 m, 0.48 m + 0.975] s. Because patches m = 0..4 lie inside [0.00, 2.895] s whereas patch m = 5 extends to 3.375 s using internal zero-padding, my sliding-window runner advances the 3.0 s buffer by **Delta_t = 2.4 s (38,400 samples)** per step and keeps only the first 5 unpadded patches (5 * 0.48 s = 2.4 s) on interior windows.

3.2 10-Class URBAN-SED to AudioSet Taxonomy Mapping

To test pretrained YAMNet on urban classes without retraining, each of the 10 URBAN-SED classes c is mapped to a set of AudioSet indices I_c in *scripts/class_mapping.py*, and combined using maximum pooling p_c(m) = max_{i in I_c} Y[m, i]. Taking the maximum avoids averaging down a strong activation on one specific subtype (such as *Medium engine* [342]). Table 1 lists the 10-class mapping.

Table 1: 10-class URBAN-SED to YAMNet (AudioSet) class mapping.

URBAN-SED Class	Alignment	YAMNet Indices (I_c)	AudioSet Display Names	Mapping Notes
air_conditioner	exact	407	Air conditioning	Direct match for air conditioning hum
car_horn	hierarchical	302, 312	Vehicle horn, car horn; Air horn, truck horn	Covers car and truck horns
children_playing	broad_proxy	66, 10	Children shouting; Children playing	Covers outdoor children vocal and play sounds
dog_bark	hierarchical	70, 69	Bark; Dog	Combines bark label with parent Dog class
drilling	exact	419	Drill	Direct match for power drills
engine_idling	hierarchical	346, 337, 342, 343	Idling; Engine; Medium engine; Heavy engine	Groups idling with parent and sibling engine nodes
gun_shot	hierarchical	421, 422	Gunshot, gunfire; Machine gun	Covers single shots and rapid gunfire

URBAN-SED Class	Alignment	YAMNet Indices (I_c)	AudioSet Display Names	Mapping Notes
jackhammer	exact	414	Jackhammer	Direct match for jackhammers
siren	hierarchical	390, 317, 318, 319, 391	Siren; Emergency vehicle; Ambulance/Police/Civil siren	Covers general and emergency vehicle sirens
street_music	broad_proxy	132, 133, 249	Music; Musical instrument; Singing	Groups street musical performance labels

3.3 Temporal Postprocessing Rules

Given a patch score sequence p_c(0), ..., p_c(M-1) spaced by Delta_p = 0.48 s, I compared four causal decision methods in *scripts/temporal_experiments.py*:

```
1. Global Thresholding (global_0.20, global_0.30): a_c(m) = 1[ p_c(m) >= theta ], theta in {0.20, 0.30} 2. Class-Specific Thresholding (class_specific): a_c(m) = 1[ p_c(m) >= theta_c^* ], tuned on Validation Split 3. Causal EMA Smoothing (ema_0.7, ema_0.5, ema_0.3): p_bar_c(m) = alpha p_c(m) + (1-alpha) p_bar_c(m-1); a_c(m) = 1[ p_bar_c(m) >= 0.20 ] 4. Dual-Threshold Hysteresis (hysteresis): a_c(m) = 1 if p_c(m) >= theta_on, or (a_c(m-1)==1 and p_c(m) >= theta_off)
```

Consecutive active patches m in [m_start, m_end] (where a_c(m) = 1) are merged into event intervals [t_hat_start, t_hat_end] = [0.48 * m_start, min(D, 0.48 * (m_end + 1))].

3.4 Validation/Test Splits, Acoustic Boundaries, and Metrics

To keep parameter tuning separate from testing, I built two sets from **ESC-50** (16 kHz mono): (1) a **Validation Split (Fold 3, 5 scenes, 56 s, 119 patches)** used only to select class thresholds (giving theta_engine^* = 0.12 and 0.20 for dog_bark, siren, car_horn) and hysteresis thresholds (giving theta_on = 0.20, theta_off = 0.06); and (2) a **Test Split (Fold 4, 12 scenes, 122 s, 260 patches, 16 audible events)** covering seven single-source scenes (*dog_only, dog_sustained, siren_only, siren_modulated, horn_only, engine_sustained, engine_fluctuating*), three gain/masking scenes (*horn_half, overlap_equal, overlap_horn_boost*), one four-class sequential scene (*sequential_4class*, 20 s), and one negative control (*silence*, 8 s).

Because ESC-50 clips often have silence inside their 5.0 s file containers (for example, *4-199261-A-0.wav* is active for 1.33 s), I measured the active start and end times [t_on, t_off] from each isolated source clip using a **-35 dB relative short-time RMS energy threshold** (25 ms frame, 10 ms hop, merging gaps < 0.45 s). I evaluate: (i) **Segment Precision, Recall, and Micro F1** over all 260 x 4 = 1,040 patch-class cells (>= 0.10 s overlap with [t_on, t_off]), (ii) **Event-Based F1** (1.0 s collar, >= 20% overlap), (iii) **Fragmentation Ratio** (predicted intervals per detected ground-truth event, ideal = 1.00), and (iv) **Mean Onset and Offset Error** (t_hat_start - t_on and t_hat_end - t_off in seconds).

4. Experimental Results

4.1 Experiment 1: Cross-Backend Numerical and Latency Check (3.0 s Window)

Table 2 compares the (6, 521) patch outputs on the 3.0 s reference clip (*yamnet/3s.wav*) across Windows x86_64 CPU, RK3588 ARM64 CPU, and RK3588 NPU Core 0 (FP16). Output files are in *results/windows_cpu/*, *results/rk3588_cpu/*, and *results/rk3588_npu/*.

Table 2: Single-window (3.0 s) numerical check and latency across backends.

Backend Comparison Pair	Output Shape	Mean Abs Diff (MAE)	Max Abs Diff	Top-1 Agreement	Single-Window Latency
Windows ONNX CPU vs. RK3588 ARM CPU	(6, 521)	4.66e-8	1.37e-6	6 / 6 patches	62.0 ms (ARM CPU)
Windows ONNX CPU vs. RK3588 NPU (FP16)	(6, 521)	1.98e-3	0.1772	5 / 6 patches	15.3 ms (NPU Core 0)
RK3588 ARM CPU vs. RK3588 NPU (FP16)	(6, 521)	1.98e-3	0.1772	5 / 6 patches	4.05x NPU speedup

4.2 Experiment 2: Initial 6-Scene Controlled Pilot Study (56 s)

In the first pilot (*scripts/pilot_detection.py*), six scenes (56 s, 119 patches) were tested on Windows CPU and RK3588 NPU using fixed thresholding (theta = 0.20) and causal EMA (alpha = 0.5). Both backends produced identical scene-level detections: **TP = 6, FP = 0, FN = 1 (100.0% Micro Precision, 85.7% Micro Recall)** (Table 3). However, Figures 1 and 2 show two issues: (1) in *sequential* (Figure 1), causal EMA (alpha = 0.5) extended every event offset by +0.48 s while the smoothed score decayed; and (2) in *overlap* (Figure 2), mixing *siren* and *car_horn* at 0.5 gain caused the NPU *car_horn* score to drop from 0.8765 (*horn_only*) to 0.0777 (0.0797 on CPU).

Table 3: Initial 6-scene pilot results on RK3588 NPU and Windows ONNX CPU.

Scene Name	Duration	Patches	Raw Intervals (theta = 0.20)	EMA Intervals (alpha = 0.5)	Detected Classes (CPU & NPU)
dog_only	8.0 s	17	1 ([0.96, 1.92] s)	1 ([0.96, 2.40] s)	dog_bark
siren_only	8.0 s	17	1 ([0.48, 6.24] s)	1 ([0.48, 6.72] s)	siren
horn_only	8.0 s	17	1 ([0.48, 3.84] s)	1 ([0.48, 4.32] s)	car_horn
sequential	14.0 s	30	2 (dog_bark, siren)	2 (dog_bark, siren)	dog_bark, siren (+0.48 s EMA tail)
overlap	10.0 s	21	1 ([0.48, 6.24] s)	1 ([0.48, 6.72] s)	siren only (car_horn missed: 0.0777)
silence	8.0 s	17	0	0	None (0 false alarms)

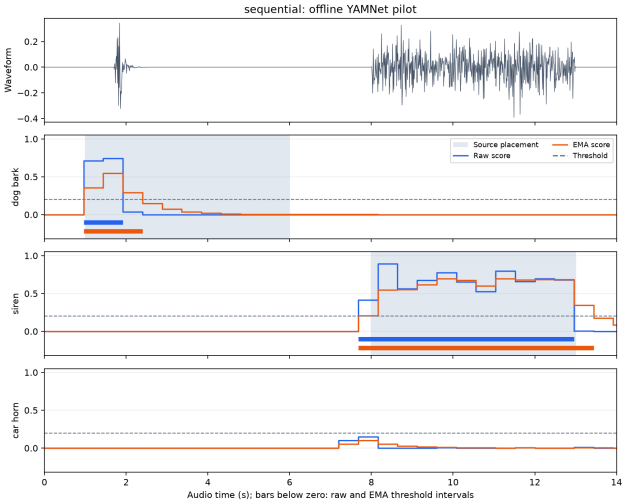


Figure 1: RK3588 NPU patch scores and detected intervals for the 14 s sequential pilot scene, showing +0.48 s offset extension under causal EMA (alpha = 0.5).

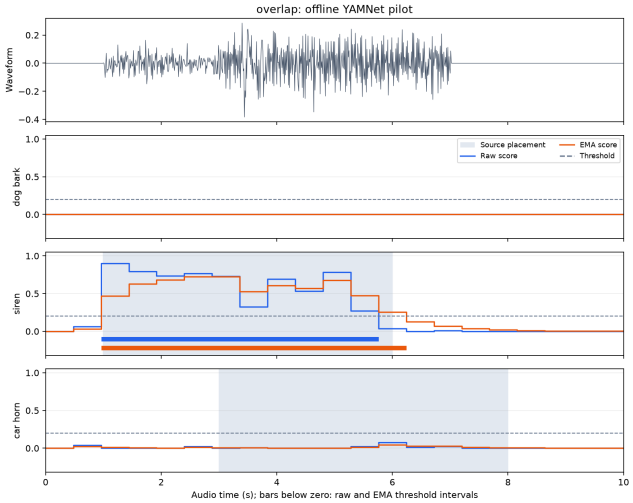


Figure 2: RK3588 NPU patch scores for the 10 s overlapping siren + car_horn pilot scene (0.5 gain each), where car_horn drops to 0.0777 below theta = 0.20.

4.3 Experiment 3: Amplitude Scaling vs. Log-Mel Spectral Masking Control

In the first pilot, both horn amplitude (1.0 -> 0.5) and siren mixing changed at the same time between *horn_only* and *overlap*. To separate these two factors, Experiment 3 tests an isolated half-gain control (*horn_half*, gain = 0.50, -6.02 dB) alongside *overlap_equal* (0.50 siren + 0.50 horn, 0 dB SIR) and *overlap_horn_boost* (0.25 siren + 0.75 horn, +9.54 dB SIR). Table 4 and Figure 3 show the results.

Table 4: Isolated -6 dB amplitude scaling control versus concurrent siren masking.

Test Scene	Siren Gain	Horn Gain	NPU Peak siren	NPU Peak car_horn	CPU Peak car_horn	Active Detection (theta = 0.20)
horn_only (unit gain)	0.00	1.00	0.0005	0.8765	0.8728	car_horn ([0.48, 3.84] s)
horn_half (-6 dB control)	0.00	0.50	0.0005	0.8296	0.8175	car_horn ([0.48, 3.84] s)
overlap_equal (0 dB SIR)	0.50	0.50	0.9004	0.0777	0.0797	siren only (car_horn masked)
overlap_horn_boost (+9.5 dB SIR)	0.25	0.75	0.9077	0.2571	0.2647	siren and car_horn (both detected)

Result: Scaling *car_horn* by 0.5 on its own (*horn_half*) lowers its NPU peak score by only **0.0469** (0.8765 -> 0.8296) and keeps the detected interval [0.48, 3.84] s unchanged. This confirms that the drop to **0.0777** in *overlap_equal* is caused by **spectral masking in the 64-band log-Mel frontend**: during [3.0, 6.0] s, the sweeping siren (500-1500 Hz) dominates the mid-frequency Mel bins shared with the horn (400-2500 Hz). Raising the horn-to-siren ratio to +9.54 dB (*overlap_horn_boost*) brings the horn peak up to 0.2571 on NPU (0.2647 on CPU), allowing both events to be detected.

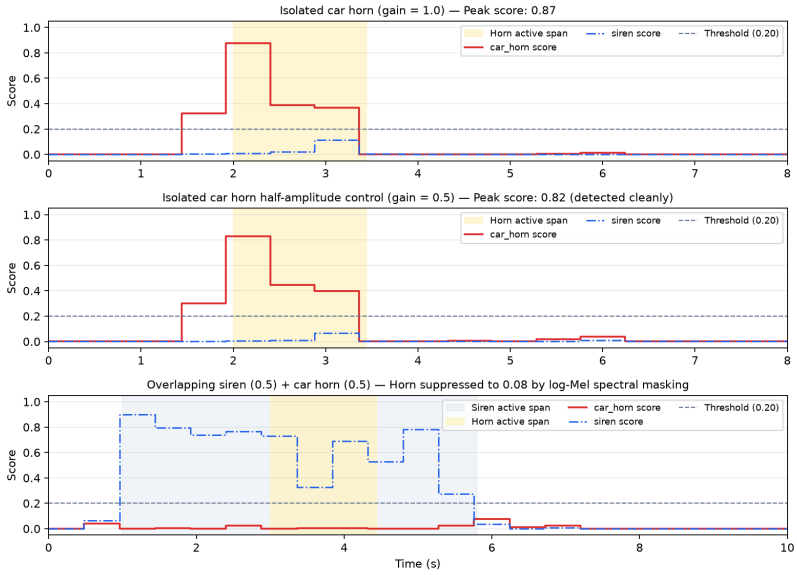


Figure 3: RK3588 NPU score trajectories comparing isolated half-gain control (horn_half, top), equal-gain overlap masking (middle), and +9.5 dB horn boost (bottom).

4.4 Experiment 4: Taxonomy Grouping on engine_idling

Testing the scenes containing *engine_idling* under *global_0.20* shows the effect of class grouping when using a 521-class model (Table 5 and Figure 4 Top). Using only *Idling* (index 346) gives 66.7% segment recall and 2.67 fragments per event because YAMNet spreads engine score across *Engine* (337) and *Medium/Heavy engine* (342, 343). Grouping [346, 337, 342, 343] via maximum pooling raises segment recall to **83.3%**, improves Segment F1 from **0.800 to 0.909**, and reduces fragmentation from **2.67 to 1.33** on both RK3588 NPU and Windows CPU.

Table 5: Single-class vs. grouped AudioSet mapping on engine_idling scenes.

Taxonomy Mapping Strategy for engine_idling	Backend	Seg Precision	Seg Recall	Seg F1	Event F1	Fragments / Event
Single Class (Idling [346] only)	RK3588 NPU	1.000	0.667	0.800	1.000	2.67
Hierarchical Engine Group ([346, 337, 342, 343])	RK3588 NPU	1.000	0.833	0.909	1.000	1.33

Taxonomy Mapping Strategy for engine_idling	Backend	Seg Precision	Seg Recall	Seg F1	Event F1	Fragments / Event
Single Class (Idling [346] only)	Windows CPU	1.000	0.667	0.800	1.000	2.67
Hierarchical Engine Group ([346, 337, 342, 343])	Windows CPU	1.000	0.833	0.909	1.000	1.33

4.5 Experiment 5: Validation Tuning and 12-Scene Test Split Comparison

Using the parameters selected on the Fold-3 Validation Split (theta_engine^* = 0.12, others 0.20; hysteresis theta_on = 0.20, theta_off = 0.06), Tables 6 and 7 compare all seven temporal decision rules across the 12-scene Fold-4 Test Split (122 s, 260 patches, 16 events) on RK3588 NPU and Windows ONNX CPU.

Table 6: 12-scene Test Split performance across seven temporal rules on RK3588 NPU (FP16).

Temporal Method	Seg P	Seg R	Seg F1	Event P	Event R	Event F1	Frag / Evt	Onset Err (s)	Offset Err (s)
global_0.20	0.949	0.746	0.835	1.000	0.882	0.938	1.53	-0.26	-0.12
global_0.30	0.976	0.648	0.779	1.000	0.882	0.938	1.47	-0.23	-0.29
class_specific	0.928	0.768	0.840	1.000	0.882	0.938	1.47	-0.29	-0.12
ema_0.7	0.907	0.761	0.828	1.000	0.882	0.938	1.33	-0.10	+0.01
ema_0.5	0.858	0.797	0.826	1.000	0.882	0.938	1.29	-0.03	+0.29
ema_0.3	0.763	0.782	0.772	0.812	0.765	0.788	1.00	+0.21	+0.66
hysteresis (0.20/0.06)	0.910	0.823	0.864	1.000	0.882	0.938	1.13	-0.26	-0.09

Table 7: 12-scene Test Split performance across seven temporal rules on Windows ONNX CPU (FP32).

Temporal Method	Seg P	Seg R	Seg F1	Event P	Event R	Event F1	Frag / Evt	Onset Err (s)	Offset Err (s)
global_0.20	0.946	0.761	0.844	1.000	0.882	0.938	1.47	-0.26	-0.12
global_0.30	0.976	0.662	0.789	1.000	0.882	0.938	1.53	-0.23	-0.29
class_specific	0.928	0.782	0.849	1.000	0.882	0.938	1.40	-0.29	-0.12
ema_0.7	0.909	0.789	0.845	1.000	0.882	0.938	1.27	-0.10	+0.07
ema_0.5	0.864	0.810	0.836	1.000	0.882	0.938	1.29	-0.03	+0.32
ema_0.3	0.767	0.789	0.778	0.812	0.765	0.788	1.00	+0.21	+0.66
hysteresis (0.20/0.06)	0.916	0.845	0.878	1.000	0.941	0.968	1.19	-0.27	+0.08

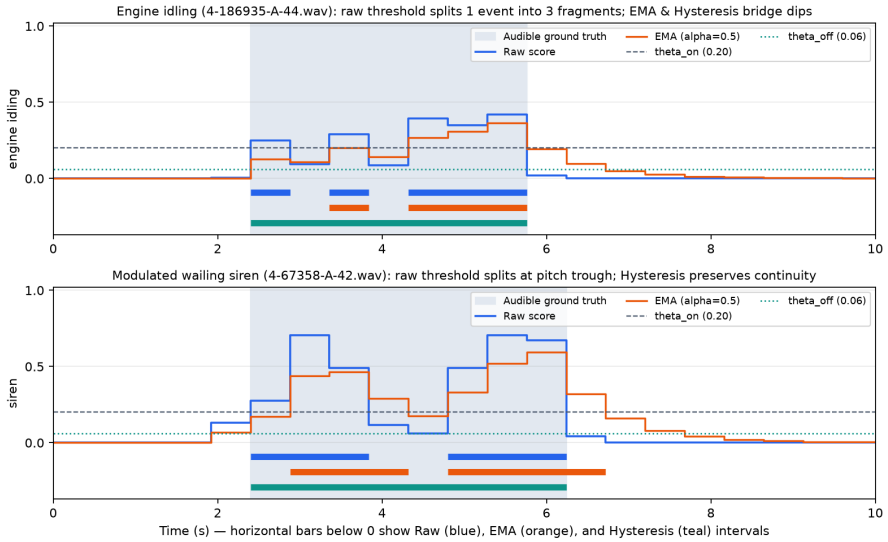


Figure 4: RK3588 NPU engine evaluation: grouped vs. single-class mapping (top), and hysteresis bridging the patch-7 dip without the offset hang-over of EMA (bottom).

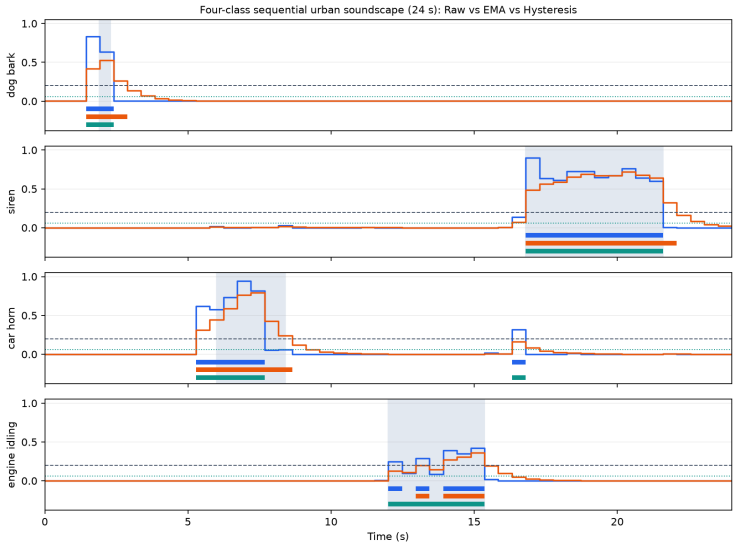


Figure 5: Four-class sequential scene (sequential_4class, 20 s) on RK3588 NPU showing clean intervals across dog_bark, car_horn, engine_idling, and siren under hysteresis.

4.6 Experiment 6: RK3588 NPU Runtime Speed

Across all 87 three-second sliding windows (178.0 s total audio across validation and test splits), I measured inference time on RK3588 NPU Core 0 (Table 8). Because each window steps forward by 2.4 s of audio and finishes in **20.88 ms** end-to-end, the detector runs at a real-time factor of **RTF = 0.00870 (about 115x faster than real time)**.

Table 8: End-to-end latency across 87 sliding windows on RK3588 NPU Core 0.

Pipeline Stage (RK3588 LubanCat-5 V2)	Mean per 3.0 s Window	Median (P50)	95th Percentile (P95)	Real-Time Factor (T / 2.4 s)
NPU Core 0 Inference (yamnet_3s.rknn, FP16)	20.83 ms	20.14 ms	26.79 ms	0.00868 (~115x real-time)
Temporal Postprocessing (EMA / Hysteresis)	< 0.05 ms	0.03 ms	0.05 ms	< 0.00002
Total End-to-End Window Execution	20.88 ms	20.17 ms	26.84 ms	0.00870

5. Discussion, Reflection, and Future Work

5.1 Why Dual-Threshold Hysteresis Outperforms Causal EMA Smoothing

Tables 6 and 7 show a clear trade-off between linear low-pass filtering (EMA) and two-threshold hysteresis. Causal EMA is a first-order linear filter with transfer function H(z) = alpha / (1 - (1 - alpha) z^-1) and time constant tau = -Delta_p / ln(1 - alpha). Because a linear filter smooths rising and falling edges in the same way, lowering alpha from 0.7 to 0.3 to bridge 1-2 patch mid-event dips (reducing fragmentation from 1.33 to 1.00) increases tau to 1.35 s. This causes two problems: (1) on short events (*dog_bark*, active duration 1.33 s), the slow rise delays onset by +0.47 s (-0.26 s -> +0.21 s) and pulls peaks below 0.20, dropping Event F1 from 0.938 to 0.788; and (2) after loud events (*siren* peaking at 0.90) stop, exponential decay keeps p_bar_c(m) above 0.20 for k = ceil(ln(0.20/0.90)/ln(1-alpha)) patches, creating a +0.66 s mean offset tail.

In contrast, **hysteresis (theta_on = 0.20, theta_off = 0.06)** uses two separate thresholds for onset and continuation. Because turning on (0 -> 1) checks the unsmoothed score p_c(m) >= theta_on directly, it adds **no filter delay at onset** (-0.26 s, matching *global_0.20*). Once active, lowering the continuation threshold to theta_off = 0.06 bridges mid-event dips on fluctuating sources (such as patch 7 of the engine clip in Figure 4 where p_engine dips to 0.069 on NPU), reducing fragmentation to 1.13 (1.00 on all single-source and sequential scenes). When the sound stops, p_c(m) falls below 0.01 within one patch, ending the interval right away (mean offset error -0.09 s on NPU) and giving the highest Segment F1 (**0.864 on NPU, 0.878 on CPU**).

5.2 Why Overlapping Sources Are Masked in Log-Mel Classifiers

Experiment 3 answered the question from the first pilot: why does *car_horn* drop from 0.8765 to 0.0777 in *overlap_equal*? Because the isolated half-gain control *horn_half* reaches 0.8296 on the NPU, -6 dB amplitude

scaling accounts for only 0.0469 of the drop. The rest of the drop comes from co-channel masking in Mel bins 12-42 (400-2500 Hz), where the continuous sweeping siren dominates E_mel[t, b] and compresses the horn's harmonic peaks under log(E_siren + E_horn + 0.001). In addition, because AudioSet clips with sirens are usually labelled mainly as *Siren*, YAMNet's 1024-D embedding layer suppresses secondary vehicle labels until the horn-to-siren ratio reaches +9.5 dB (*overlap_horn_boost*). This shows why postprocessing on p_c(m) alone cannot recover 0 dB SIR masked sources: the horn features are already masked in the log-Mel frontend.

5.3 Reflection ('If Starting Over') and Limitations

Looking back at how the project developed, there are three things I would do differently if I were starting over: (1) **Include isolated gain controls and measured -35 dB acoustic energy boundaries in the first pilot**, which would have separated amplitude scaling from spectral masking right away; (2) **Check the AudioSet class hierarchy before tuning thresholds**, as grouping the four engine labels improved Segment F1 (+0.109) much more than adjusting thresholds alone; and (3) **Keep frontend masking and backend postprocessing separate when designing tests**. Main limitations of this study are the small size of the 17-scene evaluation set (178 s total) compared to the full 2,000-clip URBAN-SED test split, the fixed 0.48 s patch hop resolution, and the fact that score-level postprocessing cannot unmix 0 dB overlapping sources.

5.4 Future Work

1. **Train a Small Classifier Head on YAMNet's (6, 1024) Embeddings:** Because *yamnet_3s.rknn* already outputs the (6, 1024) embedding tensor at no extra NPU cost, training a small linear or GRU layer on synthetic polyphonic mixtures could reduce suppression between overlapping urban sounds.
2. **Per-Channel Energy Normalisation (PCEN):** Replacing static log(Mel + 0.001) compression with adaptive PCEN (Lostanlen et al., 2018) could reduce mid-band saturation from continuous sirens and help preserve horn onsets.
3. **Background-Adaptive Thresholds:** Adjusting (theta_on, theta_off) based on a slow moving average of background scores for changing street noise levels.

6. Conclusion

This project tested an urban sound event detection pipeline on the **Rockchip RK3588 NPU** using pretrained **YAMNet**. By stepping the 3.0 s RKNN input buffer in 2.4 s unpadded hops, the pipeline matches FP32 CPU scores within 1.98e-3 MAE while running in **20.83 ms per window (RTF = 0.00868, about 115x real-time)**. Controlled tests showed that concurrent *siren* + *car_horn* suppression is caused by log-Mel spectral masking rather than -6 dB gain scaling (*horn_half* = 0.8296 vs. *overlap_equal* = 0.0777), that grouping related AudioSet engine classes raises *engine_idling* Segment F1 from 0.800 to 0.909, and that validation-tuned **dual-threshold hysteresis (theta_on = 0.20, theta_off = 0.06)** achieves the highest Test Segment F1 (**0.864 on NPU, 0.878 on CPU**) and low fragmentation (1.13) without the onset/offset delay of causal EMA smoothing.

7. References

[1] Hershey, S., Chaudhuri, S., Ellis, D. P. W., Gemmeke, J. F., Jansen, A., Moore, R. C., et al. (2017). CNN Architectures for Large-Scale Audio Classification. IEEE ICASSP 2017, pp. 131-135.

[2] Gemmeke, J. F., Ellis, D. P. W., Freedman, D., Jansen, A., Lawrence, W., Moore, R. C., et al. (2017). Audio Set: An ontology and human-labeled dataset for audio events. IEEE ICASSP 2017, pp. 776-780.

[3] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., et al. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv:1704.04861.

[4] Salamon, J., MacConnell, C., Cartwright, M., Li, P., & Bello, J. P. (2017). Scaper: A Library for Soundscape Synthesis and Augmentation. IEEE WASPAA 2017, pp. 344-348.

[5] Salamon, J., Jacoby, C., & Bello, J. P. (2014). A Dataset and Taxonomy for Urban Sound Research. Proc. ACM Multimedia 2014, pp. 1041-1044.

[6] Piczak, K. J. (2015). ESC: Dataset for Environmental Sound Classification. Proc. ACM Multimedia 2015, pp. 1015-1018.

[7] Mesaros, A., Heittola, T., & Virtanen, T. (2016). Metrics for Polyphonic Sound Event Detection. Applied Sciences, 6(6), 162.

[8] Ebbers, J., Haeb-Umbach, R., & Serizel, R. (2022). Threshold Independent Evaluation of Sound Event Detection Scores. IEEE ICASSP 2022, pp. 1021-1025.

[9] Cakir, E., Parascandolo, G., Heittola, T., Huttunen, H., & Virtanen, T. (2017). Convolutional Recurrent Neural Networks for Polyphonic Sound Event Detection. IEEE/ACM TASLP, 25(6), 1291-1303.

[10] Lostanlen, V., Salamon, J., Cartwright, M., McFee, B., Farnsworth, A., Kelling, S., & Bello, J. P. (2018). Per-Channel Energy Normalization: Why and How. IEEE Signal Processing Letters, 26(1), 39-43.

[11] Rockchip Electronics Co., Ltd. (2024). RKNN Model Zoo (examples/yamnet) and RKNN-Toolkit2 v2.3.2 Documentation. https://github.com/airockchip/rknn_model_zoo